In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Data import

In [4]:
```
df_raw_train = pd.read_csv('train_u6lujuX_CVtuZ9i.csv')
```

In [5]:
```
df_raw_test = pd.read_csv('test_Y3wMUE5_7gLdaTN.csv')
```

In [6]:
```
df_raw_train.head()
```

Out[6]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
0	LP001002	Male	No	0	Graduate	No	5849
1	LP001003	Male	Yes	1	Graduate	No	4583
2	LP001005	Male	Yes	0	Graduate	Yes	3000
3	LP001006	Male	Yes	0	Not Graduate	No	2583
4	LP001008	Male	No	0	Graduate	No	6000

In [14]:
```
df_raw_test.shape
```

Out[14]:
```
(367, 12)
```

In [8]:
```
df_raw_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   Loan_ID            614 non-null    object 
 1   Gender             601 non-null    object 
 2   Married            611 non-null    object 
 3   Dependents         599 non-null    object 
 4   Education          614 non-null    object 
 5   Self_Employed      582 non-null    object 
 6   ApplicantIncome    614 non-null    int64  
 7   CoapplicantIncome  614 non-null    float64
 8   LoanAmount         592 non-null    float64
 9   Loan_Amount_Term   600 non-null    float64
 10  Credit_History     564 non-null    float64
 11  Property_Area      614 non-null    object 
 12  Loan_Status        614 non-null    object 
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
```

In [9]:
```
df_raw_test.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   Loan_ID            367 non-null    object 
 1   Gender             356 non-null    object 
 2   Married            367 non-null    object 
 3   Dependents         357 non-null    object 
 4   Education          367 non-null    object 
 5   Self_Employed      344 non-null    object 
 6   ApplicantIncome    367 non-null    int64  
 7   CoapplicantIncome  367 non-null    int64  
 8   LoanAmount         362 non-null    float64
 9   Loan_Amount_Term   361 non-null    float64
 10  Credit_History     338 non-null    float64
 11  Property_Area      367 non-null    object 
dtypes: float64(3), int64(2), object(7)
memory usage: 34.5+ KB
```

In [16]:
```
train = df_raw_train.copy()
test = df_raw_test.copy()
```

In [23]:
```
train["Gender"].value_counts()
```

Out[23]:
```
Gender
Male      502
Female    112
Name: count, dtype: int64
```

In [21]:
```
train["Gender"].fillna("Male", inplace=True)
```

In [24]:
```
train["Married"].value_counts()
```

Out[24]:
```
Married
Yes    398
No     213
Name: count, dtype: int64
```

In [25]:
```
train["Married"].fillna("Yes", inplace=True)
```

```
C:\Users\dhruv\AppData\Local\Temp\ipykernel_18912\1365827492.py:1: FutureWarning:
A value is trying to be set on a copy of a DataFrame or Series through chained as
signment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work becau
se the intermediate object on which we are setting values always behaves as a cop
y.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.meth
od({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to pe
rform the operation inplace on the original object.


  train["Married"].fillna("Yes", inplace=True)
```

In [28]:
```
train["Dependents"].value_counts()
```

Out[28]:
```
Dependents
0     345
1     102
2     101
3+     51
Name: count, dtype: int64
```

In [29]:
```
train["Dependents"].fillna("0", inplace=True)
```

In [32]:
```
train.mode().iloc[0]
```

Out[32]:
```
Loan_ID               LP001002
Gender                    Male
Married                    Yes
Dependents                   0
Education             Graduate
Self_Employed               No
ApplicantIncome         2500.0
CoapplicantIncome          0.0
LoanAmount               120.0
Loan_Amount_Term         360.0
Credit_History             1.0
Property_Area        Semiurban
Loan_Status                  Y
Name: 0, dtype: object
```

In [37]:
```
train.isnull().sum()
```

Out[37]:
```
Loan_ID              0
Gender               0
Married              0
Dependents           0
Education            0
Self_Employed        0
ApplicantIncome      0
CoapplicantIncome    0
LoanAmount           0
Loan_Amount_Term     0
Credit_History       0
Property_Area        0
Loan_Status          0
dtype: int64
```

Based on previous rsults of the null value filling we will use mode in all

In [36]:
```
train.fillna(train.mode().iloc[0], inplace=True)
```

In [38]:
```
test.fillna(test.mode().iloc[0], inplace=True)
```

In [45]:
```
train.head()
```

Out[45]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
0	LP001002	Male	No	0	Graduate	No	584
1	LP001003	Male	Yes	1	Graduate	No	458
2	LP001005	Male	Yes	0	Graduate	Yes	300
3	LP001006	Male	Yes	0	Not Graduate	No	258
4	LP001008	Male	No	0	Graduate	No	600

In [40]:
```
test.head()
```

Out[40]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
0	LP001015	Male	Yes	0	Graduate	No	572
1	LP001022	Male	Yes	1	Graduate	No	307
2	LP001031	Male	Yes	2	Graduate	No	500
3	LP001035	Male	Yes	2	Graduate	No	234
4	LP001051	Male	No	0	Not Graduate	No	327

Encoding the Labels

In [41]:
```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
```

In [44]:
```
train["Loan_Status"] = le.fit_transform(train["Loan_Status"])
train["Loan_Status"].value_counts()
```

Out[44]:
```
Loan_Status
1    422
0    192
Name: count, dtype: int64
```

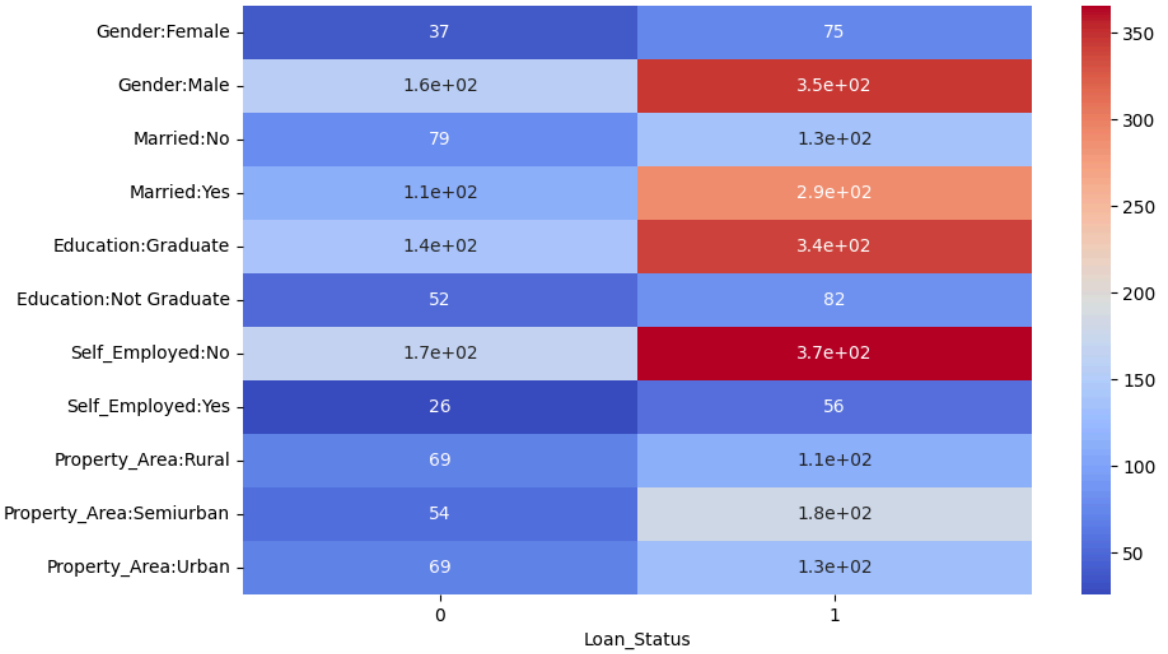
Checking the corr of the cat data with target

In [49]:
```
import seaborn as sns
features = ['Gender', 'Married', 'Education', 'Self_Employed', 'Property_Area']

# build a single table stacking crosstabs for each feature (rows labeled as "Fea
heatmap = pd.DataFrame()
for f in features:
    ct = pd.crosstab(train[f], train['Loan_Status'])
    ct.index = [f + ':' + str(idx) for idx in ct.index]
    heatmap = pd.concat([heatmap, ct])

plt.figure(figsize=(10,6))
```

```
sns.heatmap(heatmap, annot=True, cmap="coolwarm")
plt.show()
```



In [ ]:
```
from sklearn.preprocessing import OrdinalEncoder

Gender_order = ['Male', 'Female']
married_order = ['Yes', 'No']
education_order = ['Graduate', 'Not Graduate']
self_employed_order = ['No', 'Yes']
property_area_order = ['Urban', 'Semiurban', 'Rural']
oe = OrdinalEncoder(categories=[Gender_order, married_order, education_order, se
train[features] = oe.fit_transform(train[features])
```

In [ ]:
```
dependents_order = ['0', '1', '2', '3+']

dep_map = {'0': 0, '1': 1, '2': 2, '3+': 3}

orig = train['Dependents'].astype(str)
mapped = orig.map(dep_map)

if mapped.isnull().any():
        digits = orig.str.extract(r'(\d+)')[0]
        mapped.loc[mapped.isnull()] = digits.loc[mapped.isnull()].astype(float)

train['Dependents'] = mapped.astype(int)
train['Dependents'].value_counts()
```

Out[ ]:
```
Dependents
0    360
1    102
2    101
3     51
Name: count, dtype: int64
```

In [66]:
```
train.head()
```

Out[66]:

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	Coapplic
0	0.0	1.0	0	0.0	0.0	5849	
1	0.0	0.0	1	0.0	0.0	4583	
2	0.0	0.0	0	0.0	1.0	3000	
3	0.0	0.0	0	1.0	0.0	2583	
4	0.0	1.0	0	0.0	0.0	6000	

Encording all the features at once with order of high corr

In [55]:
```
from sklearn.preprocessing import OrdinalEncoder

Gender_order = ['Male', 'Female']
married_order = ['Yes', 'No']
education_order = ['Graduate', 'Not Graduate']
self_employed_order = ['No', 'Yes']
property_area_order = ['Urban', 'Semiurban', 'Rural']
oe = OrdinalEncoder(categories=[Gender_order, married_order, education_order, se
test[features] = oe.fit_transform(test[features])
```

In [56]:
```
test.head()
```

Out[56]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
0	LP001015	0.0	0.0	0	0.0	0.0	572(
1	LP001022	0.0	0.0	1	0.0	0.0	3076
2	LP001031	0.0	0.0	2	0.0	0.0	500(
3	LP001035	0.0	0.0	2	0.0	0.0	234(
4	LP001051	0.0	1.0	0	1.0	0.0	3276

In [57]:
```
train.drop(columns=['Loan_ID'], inplace=True)
test.drop(columns=['Loan_ID'], inplace=True)
```

Data split

In [72]:
```
from sklearn.model_selection import train_test_split

X = train.drop(columns=['Loan_Status'])
y = train['Loan_Status']
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_st
```

Function for the model train and evaluting

In [73]:
```
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score

results = pd.DataFrame(columns=["Model", "Accuracy", "Precision", "Recall", "F1
```

```
def train_model(model):
    model.fit(X_train, y_train)
    return model

def evaluate_model(model):
    y_val_pred = model.predict(X_val)
    print("Validation Metrics:")
    print(f"Accuracy: {accuracy_score(y_val, y_val_pred):.4f}")
    print(f"Precision: {precision_score(y_val, y_val_pred):.4f}")
    print(f"Recall: {recall_score(y_val, y_val_pred):.4f}")
    print(f"F1 Score: {f1_score(y_val, y_val_pred):.4f}")
    print(f"ROC AUC: {roc_auc_score(y_val, y_val_pred):.4f}")
    print("Confusion Matrix:")
    results.loc[len(results)] = [model.__class__.__name__,
                                 accuracy_score(y_val, y_val_pred),
                                 precision_score(y_val, y_val_pred),
                                 recall_score(y_val, y_val_pred),
                                 f1_score(y_val, y_val_pred),
                                 roc_auc_score(y_val, y_val_pred)]
    print(confusion_matrix(y_val, y_val_pred))
```

Training all the models

In [74]:
```
from sklearn.linear_model import LogisticRegression
log_reg = LogisticRegression(max_iter=1000)
log_reg = train_model(log_reg)
evaluate_model(log_reg)
```

```
Validation Metrics:
Accuracy: 0.7886
Precision: 0.7596
Recall: 0.9875
F1 Score: 0.8587
ROC AUC: 0.7031
Confusion Matrix:
[[18 25]
 [ 1 79]]
```

```
d:\College\Sem 5\ML\myenv\Lib\site-packages\sklearn\linear_model\_logistic.py:47
3: ConvergenceWarning: lbfgs failed to converge after 1000 iteration(s) (status=
1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

In [75]:
```
from sklearn.tree import DecisionTreeClassifier
dt_clf = DecisionTreeClassifier(random_state=42)
dt_clf = train_model(dt_clf)
evaluate_model(dt_clf)
```

```
Validation Metrics:
Accuracy: 0.7236
Precision: 0.7738
Recall: 0.8125
F1 Score: 0.7927
ROC AUC: 0.6853
Confusion Matrix:
[[24 19]
 [15 65]]
```

In [76]:
```python
from sklearn.neighbors import KNeighborsClassifier
knn_clf = KNeighborsClassifier()
knn_clf = train_model(knn_clf)
evaluate_model(knn_clf)
```

```
Validation Metrics:
Accuracy: 0.5772
Precision: 0.6321
Recall: 0.8375
F1 Score: 0.7204
ROC AUC: 0.4653
Confusion Matrix:
[[ 4 39]
 [13 67]]
```

In [77]:
```python
from sklearn.ensemble import RandomForestClassifier
rf_clf = RandomForestClassifier(random_state=42)
rf_clf = train_model(rf_clf)
evaluate_model(rf_clf)
```

```
Validation Metrics:
Accuracy: 0.7642
Precision: 0.7525
Recall: 0.9500
F1 Score: 0.8398
ROC AUC: 0.6843
Confusion Matrix:
[[18 25]
 [ 4 76]]
```

In [78]:
```python
from sklearn.ensemble import AdaBoostClassifier
adb_clf = AdaBoostClassifier(random_state=42)
adb_clf = train_model(adb_clf)
evaluate_model(adb_clf)
```

```
Validation Metrics:
Accuracy: 0.7886
Precision: 0.7596
Recall: 0.9875
F1 Score: 0.8587
ROC AUC: 0.7031
Confusion Matrix:
[[18 25]
 [ 1 79]]
```

In [79]:
```python
from sklearn.ensemble import GradientBoostingClassifier
gb_clf = GradientBoostingClassifier(random_state=42)
gb_clf = train_model(gb_clf)
evaluate_model(gb_clf)
```

```
Validation Metrics:
Accuracy: 0.7561
Precision: 0.7551
Recall: 0.9250
F1 Score: 0.8315
ROC AUC: 0.6834
Confusion Matrix:
[[19 24]
 [ 6 74]]
```

In [81]:
```python
from xgboost import XGBClassifier
xgb_clf = XGBClassifier(use_label_encoder=False, eval_metric='logloss',
                        random_state=42)
xgb_clf = train_model(xgb_clf)
evaluate_model(xgb_clf)
```

```
d:\College\Sem 5\ML\myenv\Lib\site-packages\xgboost\training.py:199: UserWarning:
[00:08:36] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:790:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
Validation Metrics:
Accuracy: 0.7724
Precision: 0.7708
Recall: 0.9250
F1 Score: 0.8409
ROC AUC: 0.7067
Confusion Matrix:
[[21 22]
 [ 6 74]]
```

Staking

In [83]:
```python
from sklearn.ensemble import StackingClassifier
base_estimator = [
    ('lr', log_reg),
    ('dt', dt_clf),
    ('knn', knn_clf),
]

meta_learner = RandomForestClassifier(random_state=42)
stack_clf = StackingClassifier(estimators=base_estimator, final_estimator=meta_l
stack_clf = train_model(stack_clf)
evaluate_model(stack_clf)
```

```
d:\College\Sem 5\ML\myenv\Lib\site-packages\sklearn\linear_model\_logistic.py:47
3: ConvergenceWarning: lbfgs failed to converge after 1000 iteration(s) (status=
1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
d:\College\Sem 5\ML\myenv\Lib\site-packages\sklearn\linear_model\_logistic.py:47
3: ConvergenceWarning: lbfgs failed to converge after 1000 iteration(s) (status=
1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
d:\College\Sem 5\ML\myenv\Lib\site-packages\sklearn\linear_model\_logistic.py:47
3: ConvergenceWarning: lbfgs failed to converge after 1000 iteration(s) (status=
1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
d:\College\Sem 5\ML\myenv\Lib\site-packages\sklearn\linear_model\_logistic.py:47
3: ConvergenceWarning: lbfgs failed to converge after 1000 iteration(s) (status=
1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
d:\College\Sem 5\ML\myenv\Lib\site-packages\sklearn\linear_model\_logistic.py:47
3: ConvergenceWarning: lbfgs failed to converge after 1000 iteration(s) (status=
1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
d:\College\Sem 5\ML\myenv\Lib\site-packages\sklearn\linear_model\_logistic.py:47
3: ConvergenceWarning: lbfgs failed to converge after 1000 iteration(s) (status=
1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

```
Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
Validation Metrics:
Accuracy: 0.6992
Precision: 0.7363
Recall: 0.8375
F1 Score: 0.7836
ROC AUC: 0.6397
Confusion Matrix:
[[19 24]
 [13 67]]
```

Results

In [86]:
```
results
```

Out[86]:

	Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
0	LogisticRegression	0.788618	0.759615	0.9875	0.858696	0.703052
1	DecisionTreeClassifier	0.723577	0.773810	0.8125	0.792683	0.685320
2	KNeighborsClassifier	0.577236	0.632075	0.8375	0.720430	0.465262
3	RandomForestClassifier	0.764228	0.752475	0.9500	0.839779	0.684302
4	AdaBoostClassifier	0.788618	0.759615	0.9875	0.858696	0.703052
5	GradientBoostingClassifier	0.756098	0.755102	0.9250	0.831461	0.683430
6	XGBClassifier	0.772358	0.770833	0.9250	0.840909	0.706686
7	StackingClassifier	0.699187	0.736264	0.8375	0.783626	0.639680

Knowledge Check Questions

1. What is the advantage of Random Forest over a single Decision Tree?

Random Forest reduces overfitting by combining many decision trees trained on random subsets of data and features. It gives more stable and accurate predictions compared to a single tree.

---

2. When should you use boosting instead of bagging?

Use **boosting** when you need to reduce **bias** and improve weak learners by focusing on misclassified samples. Bagging is better for reducing **variance**, while boosting aims for higher accuracy on complex patterns.

---

3. What is the role of a meta-learner in stacking?

The meta-learner learns how to best combine the predictions from base models. It assigns optimal weights or patterns to improve the final prediction performance.

---

4. How does ensemble learning reduce overfitting?

By combining multiple diverse models, ensemble learning smooths out individual model errors. This reduces the impact of noise or overfitting from any single model.

---

5. What challenges may arise while using stacking with very different base learners?

Different models may produce outputs on different scales or probabilities, causing imbalance. It also increases computational cost and risk of data leakage if not carefully cross-validated.